

Backlog API — SolarSyncBroker

EPIC-uri și story-uri de implementare, în ordinea dependențelor.

Backlog **Jira-style** pentru backend-ul `solar-sync-broker-api`, organizat pe **funcționalități** (EPIC) sparte în **stories** mici (~1 commit). Ordinea respectă dependențele din `../../PLAN_PROIECT.md`. Recap Spring: `../../RECAP_SPRING.md`. Schema DB: `db-schema.drawio`.

Arhitectura de cod = ca în `sebastian-online-school` (vezi „Arhitectură”): vertical-slice pe domeniu, servicii `command` / `query` (interfață + `Impl`), mappere statice, erori prin `ApiErrorResponse` + `ErrorConstants`, securitate pe `UserType` + `UserPermissions` cu `hasAuthority(...)`.

Nivel asumat: știi **Spring Boot**, **Data JPA**, **Security**. Noutățile — **Docker** (EPIC 14) și **Spring AI** (EPIC 10–11) — sunt marcate 📖 **înveți aici**.

Cum se citește un story-card: - `[F<epic>-S<nr>]` id · **Prioritate** (High/Med/Low) · **Points** (1/2/3/5/8, efort relativ) · **Dep** (story-uri necesare înainte). - **Ca ... vreau ... ca să ...** = de ce există story-ul. - **Acceptance (Gherkin)** = când e gata (scrii testele după scenarii). - **Tasks** = pașii pe straturi. **DoD** = checklist final (vezi și DoD global jos).

Arhitectură (model: `sebastian-online-school`)

Structura unui domeniu (vertical-slice)

```
<domeniu>/
├─ controller/      @RestController, @PreAuthorize, log.debug, ResponseEntity
├─ dtos/            Request (class @Data) + PatchRequest (record) + *Response (record)
├─ exceptions/      XNotFoundException, XAlreadyExistsException ... extends RuntimeException
├─ mapper/          XMapper – clasă cu metode STATICE toEntity / toDto
├─ model/            X – @Entity, Bean Validation, equals/hashCode/toString
├─ repository/      XRepository extends JpaRepository
├─ service/
│   ├─ commandService/ XCommandService + XCommandServiceImpl → scrieri, @Transactional
│   └─ queryService/   XQueryService + XQueryServiceImpl → citiri
```

CQRS-lite: scrierile în `commandService` , citirile în `queryService` ; fiecare interfață + `Impl` (`@Component` , injecție prin constructor).

Schelete de referință (fără comentarii în cod)

```
public interface HouseCommandService {
    HouseResponse createHouse(HouseRequest request);
}

@Component
public class HouseCommandServiceImpl implements HouseCommandService {
    private final HouseRepository houseRepository;

    public HouseCommandServiceImpl(HouseRepository houseRepository) { this.houseRepository = houseRepository; }

    @Override @Transactional
    public HouseResponse createHouse(HouseRequest request) {
        if (houseRepository.existsByName(request.getName())) throw new HouseAlreadyExistsException();
        return HouseMapper.toDto(houseRepository.save(HouseMapper.toEntity(request)));
    }
}

public class HouseMapper {
    public static House toEntity(HouseRequest r) { return House.builder()...build(); }
    public static HouseResponse toDto(House h) { return new HouseResponse(...); }
}

public class HouseNotFoundException extends RuntimeException {
    public HouseNotFoundException() { super(ErrorConstants.HOUSE_NOT_FOUND_ERROR); }
}

@RestController @RequestMapping("/api/v1/houses") @Slf4j
public class HouseController {
    private final HouseCommandService commandService;
    private final HouseQueryService queryService;

    public HouseController(HouseCommandService c, HouseQueryService q) { this.commandService = c; this.queryService = q; }

    @PostMapping @PreAuthorize("hasAuthority('HOUSE_MANAGE')")
    public ResponseEntity<HouseResponse> create(@Valid @RequestBody HouseRequest request) {
        log.debug("http post /api/v1/houses");
        return ResponseEntity.status(HttpStatus.CREATED).body(commandService.createHouse(request));
    }
}
```

Cross-cutting (`system/` + `auth/` + `users/`)

```
system/    config/ (OpenApiConfig, ClockConfig, *Properties) · constants/ (ErrorConstants, HintsCo
auth/      controller/ · dtos/ · exceptions/ · service/ (AuthService + AuthServiceImpl)
users/     dtos/ · exceptions/ · jwt/ (JWTTokenProvider) · mapper/ · model/ (User implements UserD
```

Erori — `ApiErrorResponse` , NU `ProblemDetail`

```
@Builder
public record ApiErrorResponse(String timestamp, int status, String error, String message, String
```

`GlobalExceptionHandler` (`@RestControllerAdvice`) grupează pe status: `*NotFoundException` →404, `*AlreadyExistsException` →409, validare→400/422, fallback→500. Mesaje din `ErrorConstants` , în engleză.

Securitate — `UserType` + `UserPermissions` + `hasAuthority`

- `User implements UserDetails` cu `UserType` (enum) și `Set<UserPermissions>` (`@ElementCollection` , tabel `user_permissions`); `getAuthorities()` din permisiuni.
- token JWT emis cu `JWTTokenProvider` (jjwt, HS512, claim `authorities`), **validat** prin `oauth2ResourceServer().jwt(NimbusJwtDecoder)` .
- `SecurityConfiguration` : stateless, `@EnableMethodSecurity` , `DaoAuthenticationProvider` + `BCryptPasswordEncoder` , `PUBLIC_URLS` + `publicAwareBearerTokenResolver` .
- pe endpoint: `@PreAuthorize("hasAuthority('...')")` . Config:
`application.jwt.secretKey=${JWT_SECRET_KEY}` .

Matricea de permisiuni (`UserPermissions`)

Permisiune	ADMIN	OPERATOR	USER	unde
<code>HOUSE_VIEW</code>	✓	✓	✓ ¹	GET case/state
<code>HOUSE_MANAGE</code>	✓			POST/PATCH case
<code>BATTERY_VIEW</code>	✓	✓	✓ ¹	GET baterie
<code>BATTERY_CONFIG</code>	✓			PATCH config
<code>BATTERY_COMMAND</code>	✓	✓		comenzi baterie
<code>SIMULATION_CONTROL</code>	✓			start/pause/stop/reset
<code>TELEMETRY_VIEW</code>	✓	✓	✓ ¹	GET telemetrie
<code>COMMUNITY_VIEW</code>	✓	✓		GET community/*
<code>MARKET_VIEW</code>	✓	✓	✓	GET preț/istoric
<code>MARKET_MANAGE</code>	✓			POST scenarii
<code>GRID_VIEW</code>	✓	✓		GET grid/alerte
<code>FINANCIAL_VIEW</code>	✓	✓	✓ ¹	GET financial-summary
<code>AGENT_VIEW</code>	✓	✓		GET status/runs/plans
<code>AGENT_CONFIG</code>	✓			PATCH agent config
<code>AGENT_OPERATE</code>	✓	✓		run/approve/reject/execute/emergency-stop
<code>AUDIT_VIEW</code>	✓	✓		GET audit/rapoarte

¹ `USER` are permisiunea dar vede **doar casa proprie** — owner check **în service** (`getXForCaller`), nu din adnotare.

Diferit față de online-school (intenționat)

Aspect	online-school	solar	de ce
Schema DB	ddl-auto: update	Flyway + validate	versionat, reproductibil
Prefix API	/api/v2	/api/v1	PLAN
Bani	—	BigDecimal	corectitudine financiară

Teste (layout online-school)

.../<domeniu>Test/	XCommandServiceImplTest, XQueryServiceImplTest	(unit, Mockito)
.../controllerTest/	xControllerTest	(MockMvc standalone)
.../integration/...	XControllerTests	(Testcontainers MySQL)

Harta EPIC-urilor

EPIC	Funcționalitate	Stories	Dep	PLAN
00	Fundație	S1–S3	—	1
01	Autentificare & permisiuni	S1–S3	00	2
02	Adminul gestionează case	S1–S3	01	2
03	Configurarea bateriilor	S1–S2	02	2
04	Simularea produce telemetrie	S1–S4	03	3
05	Bilanț energetic & SoC	S1–S3	04	4
06	Comenzi manuale de baterie	S1	05	4
07	Prețul energiei (piață)	S1–S2	04	4
08	Protecția transformatorului	S1–S3	05,07	5
09	Contabilitate financiară	S1–S2	05,07	4
10	 Agent: runs & audit (ADVISORY)	S1–S3	05,08	6
11	 Agent: plan validat	S1–S3	10	6–7
12	Audit & rapoarte	S1	toate	10
13	Dashboard live (WebSocket)	S1	04,08,10	9
14	 Rulare în Docker Compose	S1–S2	01+	1/10

Oprește-te la **EPIC 11** în mod **ADVISORY** pentru licență. *Modelul propune, codul determinist dispune.*

EPIC 00 — Fundație

[F00–S1] Erori unitare cu `ApiResponse`

Prioritate: High · Points: 3 · Dep: —

Ca dezvoltator, **vreau** un format de eroare unic pe toată aplicația, **ca să** clientul primească mereu același JSON și să nu scape stack trace-uri.

Acceptance (Gherkin):

Scenario: excepție de business

Given un endpoint care aruncă o `*NotFoundException`

When clientul îl apelează

Then primește 404 cu `ApiResponse` (timestamp, status, error, message, path)

Scenario: validare eșuată

Given un body invalid (@Valid pică)

When POST pe acel endpoint

Then 422 cu error="VALIDATION_ERROR" și câmpurile invalide

Scenario: eroare neașteptată

Given o excepție necontrolată

Then 500 fără stack trace în body

Tasks: - [] `system/exceptions/ApiResponse` (record `@Builder`) - []
`system/constants/ErrorConstants` (mesaje `static final String`, EN) - []
`system/exceptions/GlobalExceptionHandler` (`@RestControllerAdvice`, grupat pe status) - []
controller dummy + test care verifică cele 3 scenarii

DoD: compile verde · texte EN · fără stack trace în body.

[F00-S2] OpenAPI / Swagger cu Bearer

Prioritate: Med · **Points:** 1 · **Dep:** F00-S1

Ca dezvoltator/comisie, **vreau** documentația API live, **ca să** explorez endpoint-urile și să testez cu token.

Acceptance (Gherkin):

Scenario: Swagger accesibil

When deschid `/swagger-ui.html`

Then se încarcă cu titlul "SolarSyncBroker API" și schema `bearerAuth`

Tasks: - [] `system/config/OpenApiConfig` (`@Bean OpenAPI`, security scheme `bearerAuth`)

DoD: Swagger se încarcă · butonul Authorize există.

[F00-S3] Config tipizată, auditing & clock

Prioritate: High · **Points:** 2 · **Dep:** F00-S1

Ca dezvoltator, **vreau** parametrii fizici tipizați și timpul injectabil, **ca să** evit valori magice și să fac testele deterministe.

Acceptance (Gherkin):

```
Scenario: properties încărcate
  Given application.yml cu solarsync.simulation.* / grid.* / agent.*
  When pornește aplicația
  Then SimulationProperties, GridProperties, AgentProperties sunt injectabile

Scenario: timp controlabil în test
  Given un Clock.fixed injectat
  Then logica datează evenimente cu timpul fix
```

Tasks: - [] `@EnableJpaAuditing` + `AbstractAuditable` (`@CreateDate` `Instant createdAt`) - [] `SimulationProperties` (`seed` , `tickSeconds` , `realToVirtualRatio`) - [] `GridProperties` (`transformerNominalKw` , `safetyMarginPercent`) - [] `AgentProperties` (`mode` default `ADVISORY` , `planningIntervalSeconds`) - [] `ClockConfig` (`@Bean` `Clock`) — nicăieri `Instant.now()` direct - [] chei în `application.yml` (fără secrete)

DoD: compile verde · valori implicite prezente.

EPIC 01 — Autentificare & permisiuni

[F01-S1] User, UserType & permisiuni (DB + model)

Prioritate: High · **Points:** 3 · **Dep:** F00-S3

Ca sistem, **vreau** utilizatori cu tip și set de permisiuni, **ca să** pot autoriza pe `hasAuthority` .

Acceptance (Gherkin):

Scenario: schema creată

When pornește Flyway

Then există tabelele users și user_permissions, iar ddl-auto:validate trece

Scenario: authorities din permisiuni

Given un User cu permisiunile [HOUSE_VIEW, MARKET_VIEW]

Then getAuthorities() le întoarce ca SimpleGrantedAuthority

Tasks: - [] V1__users.sql : users + user_permissions - [] users/model/User implements UserDetails , UserType (ADMIN/OPERATOR/USER/AGENT) - [] users/security/UserPermissions (enum din matrice) - [] users/repository/UserRepository (findByEmail , existsByEmail)

DoD: Flyway aplică V1 · validate trece.

[F01-S2] Security config + emitere/validare JWT

Prioritate: High · **Points:** 5 · **Dep:** F01-S1

Ca sistem, **vreau** API stateless securizat cu JWT, **ca să** doar cererile autentificate ajungă la business.

Acceptance (Gherkin):

Scenario: rută protejată fără token

When GET /api/v1/houses fără Authorization

Then 401 cu ApiErrorResponse

Scenario: rute publice

When POST /api/v1/auth/login

Then nu cere token (permitAll)

Scenario: permisiune lipsă

Given token valid fără HOUSE_MANAGE

When POST /api/v1/houses

Then 403 cu ApiErrorResponse

Tasks: - [] SecurityConstants (AUTHORITIES , ISSUER , AUDIENCE , PUBLIC_URLS) - [] users/jwt/JWTTokenProvider (HS512, claim authorities , cheia din env) - [] users/service/UserDetailsService(+Impl) - [] SecurityConfiguration (stateless, @EnableMethodSecurity , BCrypt , DaoAuthenticationProvider , oauth2ResourceServer().jwt , publicAwareBearerTokenResolver) - [] JwtAuthenticationEntryPoint (401) + SecurityAccessDeniedHandler (403) → ApiErrorResponse

DoD: 401/403 ies ca `ApiResponse` · rute publice merg fără token.

[F01-S3] Register / Login / Me

Prioritate: High · **Points:** 3 · **Dep:** F01-S2

Ca utilizator, **vreau** să mă înregistrez și să mă loghez, **ca să** primesc un token și să-mi văd profilul.

Acceptance (Gherkin):

```
Scenario: înregistrare
  When POST /auth/register cu email nou
  Then 201, parola e hash-uită, primesc permisiunile UserType-ului

Scenario: email duplicat
  Given email deja folosit
  When POST /auth/register
  Then 409 UserAlreadyExistsException

Scenario: login + me
  Given user existent
  When POST /auth/login apoi GET /auth/me cu Bearer
  Then login→200+token, me→200 cu datele mele
```

Tasks: - [] DTO `auth/dtos/` : `AuthRegisterRequest` , `AuthLoginRequest` , `AuthLoginResponse` - []
`auth/service/AuthService(+Impl)` : `register` (set permisiuni după tip), `login`
(`AuthenticationManager`) - [] `auth/exceptions/AuthValidationException` ;
`users/exceptions/User*Exception` - [] `auth/controller/AuthController` (register/login/me/logout) -
[] test integrare: register→login→me; 401 fără token

DoD: flux complet verde · permisiuni corecte per tip.

EPIC 02 — Adminul gestionează case

[F02-S1] Crearea unei case

Prioritate: High · **Points:** 3 · **Dep:** F01-S3

Ca ADMIN, **vreau** să creez o casă cu parametrii fizici, **ca să** intre în simulare.

Acceptance (Gherkin):

Scenario: creare validă

Given autentificat cu HOUSE_MANAGE

When POST /api/v1/houses cu puteri > 0

Then 201 și casa apare în GET /api/v1/houses

Scenario: fără permisiune

Given autentificat fără HOUSE_MANAGE

When POST /api/v1/houses

Then 403 ApiErrorResponse

Scenario: putere invalidă

When POST cu pv_peak_power_kw <= 0

Then 422 VALIDATION_ERROR

Tasks: - [] V2__houses.sql - [] house/model/House + HouseRepository (existsByName ,
findByOwnerId , findByIdAndOwnerId) - [] DTO HouseRequest (@Positive), HouseResponse +
HouseMapper static - [] HouseCommandServiceImpl.createHouse - [] POST /houses
@PreAuthorize('HOUSE_MANAGE') - [] test 201 + 403 + 422

DoD: Flyway V2 · DoD global.

[F02-S2] Vizualizare cu izolare pe owner

Prioritate: High · **Points:** 3 · **Dep:** F02-S1

Ca USER, **vreau** să-mi văd doar casa mea; **ca** ADMIN, toate, **ca să** datele altcuiva rămână private.

Acceptance (Gherkin):

```
Scenario: admin vede tot
  Given ADMIN
  When GET /api/v1/houses
  Then 200 cu toate casele
```

```
Scenario: owner vede casa lui
  Given USER owner al casei {id}
  When GET /api/v1/houses/{id}
  Then 200
```

```
Scenario: user pe casa altuia
  Given USER care nu deține {altId}
  When GET /api/v1/houses/{altId}
  Then 403 HouseAccessDeniedException
```

Tasks: - [] DTO `HouseStateResponse` - [] `HouseQueryService` : `getAllHouses()` , `getHouseForCaller(id, caller)` (owner check **în service**) - [] `GET /houses` , `/houses/{id}` , `/houses/{id}/state` `@PreAuthorize('HOUSE_VIEW')` - [] `HouseNotFoundException` (404), `HouseAccessDeniedException` (403) - [] test: 2 useri, owner 200 / străin 403

DoD: owner check verificat prin test.

[F02-S3] Update / Patch / Delete

Prioritate: Med · **Points:** 2 · **Dep:** F02-S1

Ca ADMIN, **vreau** să modific sau să șterg o casă, **ca să** întrețin comunitatea.

Acceptance (Gherkin):

```
Scenario: patch parțial
  Given casa {id}
  When PATCH /houses/{id} cu un singur câmp
  Then 200, doar câmpul trimis se schimbă
```

```
Scenario: casă inexistentă
  When PATCH /houses/{lipsă}
  Then 404 HouseNotFoundException
```

Tasks: - [] DTO `HousePatchRequest` (record) - [] `HouseCommandService` : `updateHouse` , `patchHouse` , `deleteHouse` - [] `PATCH /houses/{id}` `@PreAuthorize('HOUSE_MANAGE')` - [] test patch + 404

DoD: DoD global.

EPIC 03 — Configurarea bateriilor

[F03-S1] Bateria casei (entity + GET)

Prioritate: High · **Points:** 3 · **Dep:** F02-S1

Ca owner/ADMIN, **vreau** să văd bateria casei (SoC + limite), **ca să** știu starea ei.

Acceptance (Gherkin):

```
Scenario: citire baterie
  Given casa {id} cu baterie
  When GET /api/v1/houses/{id}/battery cu BATTERY_VIEW
  Then 200 cu SoC, limite, randamente

Scenario: owner check
  Given USER care nu deține casa
  Then 403
```

Tasks: - [] `V3__batteries.sql` (cu `version`) - [] `battery/model/Battery` (`@Version`) + `BatteryRepository.findById` - [] DTO `BatteryResponse` + mapper static - [] `BatteryQueryService` (owner check) - [] `GET /houses/{houseId}/battery` `@PreAuthorize('BATTERY_VIEW')` - [] test GET + 403

DoD: Flyway V3 · `@Version` prezent.

[F03-S2] Configurarea bateriei

Prioritate: Med · **Points:** 2 · **Dep:** F03-S1

Ca ADMIN, **vreau** să modific limitele bateriei, **ca să** ajustez comportamentul fizic.

Acceptance (Gherkin):

```
Scenario: patch config
  Given BATTERY_CONFIG
  When PATCH /battery/configuration cu maxChargePowerKw nou
  Then 200 și valoarea se schimbă

Scenario: fără permisiune
  Given doar BATTERY_VIEW
  Then 403
```

Tasks: - [] DTO `BatteryConfigurationRequest` - [] `BatteryCommandService.updateConfiguration` - [] `PATCH /battery/configuration @PreAuthorize('BATTERY_CONFIG')` - [] test patch + 403

DoD: DoD global.

EPIC 04 — Simularea produce telemetrie

[F04-S1] Entitatea Telemetry + citire

Prioritate: High · **Points:** 3 · **Dep:** F03-S1

Ca owner/ADMIN, **vreau** istoricul de telemetrie, **ca să** urmăresc casa în timp.

Acceptance (Gherkin):

```
Scenario: telemetrie paginată
  Given telemetrie pentru casa {id}
  When GET /houses/{id}/telemetry?page=0
  Then 200 cu pagină ordonată desc după recorded_at
```

Tasks: - [] `V4__telemetry.sql` + index `(house_id, recorded_at)` - [] `telemetry/model` + repo `(findByHouseIdOrderByRecordedAtDesc, paginat)` - [] DTO `TelemetryResponse`, `HouseTelemetryPageResponse`

DoD: index creat · Flyway V4.

[F04-S2] Generatoare PV & consum deterministe

Prioritate: High · **Points:** 3 · **Dep:** F00-S3

Ca sistem, **vreau** producție/consum reproductibile pe seed, **ca să** testele dea același rezultat.

Acceptance (Gherkin):

```
Scenario: reproductibilitate
  Given același seed, casă și timp
  When apelez PvGenerator/LoadGenerator de 2 ori
  Then primesc valori identice
```

Tasks: - [] `simulation/service/PvGenerator`, `LoadGenerator` — `powerKw(House, SimulationTime, long seed)` - [] unit test reproductibilitate

DoD: test verde · fără randomness necontrolat.

[F04-S3] Tick central + lifecycle

Prioritate: High · **Points:** 5 · **Dep:** F04-S1, F04-S2

Ca ADMIN, **vreau** să pornesc/opresc simularea, **ca să** controlez comunitatea, cu **un singur tick** care actualizează toate casele.

Acceptance (Gherkin):

```
Scenario: pornire produce date
  Given SIMULATION_CONTROL
  When POST /simulation/start și trec câteva tick-uri
  Then telemetria caselor se populează

Scenario: reset determinist
  When POST /simulation/reset
  Then simularea reia cu același seed

Scenario: un singur tick
  Then toate casele sunt actualizate de un tick central (nu 10 fire)
```

Tasks: - [] `SimulationClock` (timp virtual cu `tickSeconds`) - [] `SimulationEngine.tick()` (PV+consum, netPower provizoriu, persistă telemetrie) - [] state machine `STOPPED→RUNNING→PAUSED` - [] `SimulationController` start/pause/resume/stop/reset/config `@PreAuthorize('SIMULATION_CONTROL')`, status authenticated - [] test lifecycle + reset

DoD: un tick central · reset reproductibil.

[F04-S4] Telemetrie & stare comunitate

Prioritate: Med · **Points:** 2 · **Dep:** F04-S3

Ca ADMIN/OPERATOR, **vreau** starea comunității, **ca să** văd ansamblul.

Acceptance (Gherkin):

```
Scenario: community state
  Given COMMUNITY_VIEW
  When GET /community/state
  Then 200 cu agregarea caselor
```

Tasks: - [] GET /houses/{id}/telemetry @PreAuthorize('TELEMETRY_VIEW') + owner check - [] GET /community/telemetry , /community/state @PreAuthorize('COMMUNITY_VIEW') - [] test

DoD: DoD global.

EPIC 05 — Bilanț energetic & SoC (inima deterministă)

[F05-S1] Energy engine

Prioritate: High · **Points:** 5 · **Dep:** F04-S3

Ca sistem, **vreau** bilanțul energetic corect, **ca să** importul/exportul și energia pe tick fie exacte (PLAN 7.1).

Acceptance (Gherkin):

```
Scenario: convenția de semn
  Given Ppv, Pload, Pbatt
  When compute(...)
  Then Pnet = Ppv - Pload + Pbatt; Pnet>0 export, Pnet<0 import

Scenario: bilanț închis
  Then energia se conservă în toleranța numerică
```

Tasks: - [] `energy/service/EnergyEngine.compute(houseState, batteryCommand, deltaHours)` - []

unit: bilanț, conversie putere→energie, semn export/import

DoD: teste verzi — **blocant** pentru EPIC-urile următoare.

[F05-S2] Actualizare SoC + constrângeri

Prioritate: High · **Points:** 5 · **Dep:** F05-S1, F03-S1

Ca sistem, **vreau** SoC actualizat cu randament și limitat fizic, **ca să** bateria nu iasă din `[minSoC, maxSoC]` (PLAN 7.2).

Acceptance (Gherkin):

Scenario: încărcare/descărcare

Then SoC se schimbă conform formulei, cu randamentul aplicat

Scenario: clamp limite

Given comandă care ar trece de maxSoC sau sub minSoC

Then SoC e limitat / comanda respinsă

Tasks: - [] `battery/service/commandService/BatteryService.applyCommand` - [] unit: charge, discharge, peste `maxChargePowerKw` respinsă, sub `minSoC` respinsă, randamente

DoD: SoC nu iese niciodată din limite (test).

[F05-S3] Validator determinist de comandă

Prioritate: High · **Points:** 3 · **Dep:** F05-S2

Ca sistem, **vreau** un singur validator pentru orice comandă (manuală sau de agent), **ca să** regulile fizice fie aplicate uniform.

Acceptance (Gherkin):

Scenario: comandă peste putere

When validez o comandă cu powerKw > limită

Then respinsă cu motiv structurat (BATTERY_POWER_LIMIT)

Scenario: comandă infeasibilă SoC

Then respinsă (BATTERY_SOC_LIMIT)

Tasks: - [] battery/service/BatteryCommandValidator - [] BatterySocLimitException ,
BatteryPowerLimitException (422, mesaj din ErrorConstants) - [] unit pe ambele respingeri

DoD: reutilizabil de F06 și F11.

EPIC 06 — Comenzi manuale de baterie

[F06-S1] Trimitere comandă validată

Prioritate: High · **Points:** 3 · **Dep:** F05-S3

Ca ADMIN/OPERATOR, **vreau** să trimit o comandă de baterie, **ca să** intervin manual — prin **același** validator ca agentul.

Acceptance (Gherkin):

```
Scenario: comandă validă
  Given BATTERY_COMMAND
  When POST /houses/{id}/battery/commands valid
  Then 201, SoC actualizat, CommandExecution persistat

Scenario: comandă peste limită
  Then 422 cu ApiErrorResponse (errorCode)
```

Tasks: - [] V5__command_execution.sql + agent/model/CommandExecution - [] DTO
BatteryCommandRequest , CommandExecutionResponse - [] GET / POST
/houses/{id}/battery/commands @PreAuthorize('BATTERY_COMMAND') - [] test 201 + 422

DoD: trece prin BatteryCommandValidator .

EPIC 07 — Prețul energiei (piață)

[F07-S1] MarketPrice + simulator

Prioritate: High · **Points:** 3 · **Dep:** F00-S3

Ca sistem, **vreau** preț pe intervale virtuale cu scenarii, **ca să** simulez piața (mic/negativ/ridicat).

Acceptance (Gherkin):

```
Scenario: preț curent
  When cer prețul intervalului virtual curent
  Then primesc o valoare conform scenariului activ
```

Tasks: - [] `V6__market_price.sql` + `market/model/MarketPrice` - []
`market/service/MarketSimulator` - [] unit scenarii

DoD: Flyway V6.

[F07-S2] Endpoints piață + integrare în tick

Prioritate: Med · **Points:** 2 · **Dep:** F07-S1, F04-S3

Ca utilizator, **vreau** prețul și istoricul; **ca** sistem, prețul intră în telemetrie.

Acceptance (Gherkin):

```
Scenario: istoric paginat
  When GET /market/prices cu MARKET_VIEW
  Then 200 paginat

Scenario: scenariu nou
  Given MARKET_MANAGE
  When POST /market/scenarios
  Then se schimbă scenariul; USER fără permisiune → 403

Scenario: preț în telemetrie
  Then fiecare tick scrie market_price în telemetry
```

Tasks: - [] `GET /market/current-price`, `/market/prices` `@PreAuthorize('MARKET_VIEW')` - [] `POST /market/scenarios` `@PreAuthorize('MARKET_MANAGE')` - [] integrare în `tick()` - [] test

DoD: telemetria primește prețul.

EPIC 08 — Protecția transformatorului

[F08-S1] Snapshot grid + detecție suprasarcină

Prioritate: High · **Points:** 3 · **Dep:** F05-S1

Ca sistem, **vreau** să detectez depășirea limitei transformatorului, **ca să** protejez infrastructura.

Acceptance (Gherkin):

```
Scenario: limita operațională
  Given nominal=50kW, safetyMargin=10%
  Then operationalLimit=45kW

Scenario: detecție depășire
  Given flux total > operationalLimit
  Then se semnalează suprasarcină (import sau export)
```

Tasks: - [] `V7__grid.sql` (`grid_snapshot` , `grid_alert`) - [] `grid/service/GridProtectionService` (limită + detecție) - [] unit detecție

DoD: Flyway V7.

[F08-S2] Curtailment determinist

Prioritate: High · **Points:** 5 · **Dep:** F08-S1

Ca sistem, **vreau** reducere proporțională la suprasarcină, **ca să** fluxul scadă sub limită — cu **prioritate față de agent**.

Acceptance (Gherkin):

```
Scenario: 60kW export, limită 45kW
  When grid protection rulează înainte de aplicarea comenzilor
  Then exportul final ≤ 45kW (reducere proporțională)
  And se creează o alertă auditată
```

Tasks: - [] curtailment proporțional în `GridProtectionService` - [] integrare în `tick()` înainte de aplicare - [] unit dedicat 60→45 + alertă

DoD: scenariul PLAN 15.4 verde.

[F08-S3] Endpoints grid

Prioritate: Med · **Points:** 2 · **Dep:** F08-S1

Ca ADMIN/OPERATOR, **vreau** starea și alertele grid-ului, **ca să** monitorizez.

Acceptance (Gherkin):

```
Scenario: listă alerte
  When GET /grid/alerts cu GRID_VIEW
  Then 200
```

Tasks: - [] GET /grid/state , /grid/alerts , /grid/alerts/{id} @PreAuthorize('GRID_VIEW') - [] test

DoD: DoD global.

EPIC 09 — Contabilitate financiară

[F09-S1] FinancialTransaction la fiecare tick

Prioritate: High · **Points:** 3 · **Dep:** F05-S1, F07-S1

Ca sistem, **vreau** să înregistrez BUY/SELL la fiecare tick, **ca să** pot calcula profitul.

Acceptance (Gherkin):

```
Scenario: import = BUY, export = SELL
  Given un tick cu import și prețul curent
  Then se creează o tranzacție BUY cu amount = energy_kwh * unit_price (BigDecimal)
```

Tasks: - [] V8__financial.sql + financial/model/FinancialTransaction (BigDecimal) - []
generare în tick() - [] unit calcul

DoD: bani cu BigDecimal .

[F09-S2] Sumarele financiare

Prioritate: Med · **Points:** 2 · **Dep:** F09-S1

Ca owner/ADMIN, **vreau** cost/venit/profit, **ca să** evaluez casa și comunitatea.

Acceptance (Gherkin):

Scenario: profit seara

Given scenariul "preț ridicat seara" (PLAN 15.4)

Then profitul calculat e corect

Scenario: izolare

Given USER

When GET /houses/{altId}/financial-summary

Then 403

Tasks: - [] GET /houses/{id}/financial-summary @PreAuthorize('FINANCIAL_VIEW') + owner check
- [] GET /community/financial-summary @PreAuthorize('COMMUNITY_VIEW') - [] test profit + izolare

DoD: scenariul PLAN 15.4 verde.

EPIC 10 — Agent: runs & audit (ADVISORY)

 **Înveți aici — Spring AI, pasul 1.** Abstractizezi LLM-ul după `AgentModelGateway`. Mock determinist în teste, fără rețea/cheie. Codul Spring AI real apare la Modul 8, izolat. **Agentul nu e chatbot.**

[F10-S1] Entități de audit ale agentului

Prioritate: High · **Points:** 3 · **Dep:** F05-S2

Ca sistem, **vreau** să persist run-urile, planurile și tool-call-urile, **ca să** fiecare decizie a agentului fie auditabilă.

Acceptance (Gherkin):

Scenario: schema agent

When Flyway aplică V9

Then există agent_run, agent_plan, agent_tool_call

Tasks: - [] `V9__agent.sql` (3 tabele, câmpuri din PLAN 9) - [] `agent/model/` + `agent/repository/`

DoD: Flyway V9 · validate trece.

[F10-S2] Gateway LLM (abstracție + mock)

Prioritate: High · **Points:** 3 · **Dep:** F10-S1

Ca dezvoltator, **vreau** o singură interfață spre model, **ca să** testez agentul fără LLM real.

Acceptance (Gherkin):

```
Scenario: mock determinist
  Given AgentModelGateway mock
  When proposePlan(ctx)
  Then întoarce un DispatchPlan fix, fără apel de rețea
```

Tasks: - [] `agent/service/AgentModelGateway` (interfață) — `DispatchPlan proposePlan(AgentContext)` - [] implementare mock - [] note Modul 8: provider OpenAI, `AI_PROVIDER` + cheie din env, aceeași interfață

DoD: zero dependență de rețea în teste.

[F10-S3] Status, runs & trigger manual

Prioritate: Med · **Points:** 3 · **Dep:** F10-S2

Ca ADMIN/OPERATOR, **vreau** să declanșez și să inspectez run-uri, **ca să** văd ce face agentul.

Acceptance (Gherkin):

```
Scenario: trigger creează run
  Given AGENT_OPERATE
  When POST /agent/run
  Then se creează AgentRun și se persistă AgentToolCall-urile

Scenario: config doar admin
  Given OPERATOR (fără AGENT_CONFIG)
  When PATCH /agent/configuration
  Then 403
```

Tasks: - [] `GET /agent/status`, `/agent/runs`, `/runs/{id}` `@PreAuthorize('AGENT_VIEW')` - [] `PATCH /agent/configuration` `@PreAuthorize('AGENT_CONFIG')` - [] `POST /agent/run` `@PreAuthorize('AGENT_OPERATE')` - [] test trigger + audit

DoD: run-ul e auditat complet.

EPIC 11 — Agent: plan validat (ADVISORY)

 **Înveți aici — Spring AI, pasul 2.** Structured output (`DispatchPlan` , nu text). Orice plan trece prin **același** `BatteryCommandValidator` + check transformator. *Modelul propune, codul dispune.* Tool calling la Modul 8.

[F11-S1] DispatchPlan (structured output)

Prioritate: High · **Points:** 3 · **Dep:** F10-S2

Ca sistem, vreau un plan structurat și validabil, **ca să** nu execut text liber.

Acceptance (Gherkin):

```
Scenario: plan valid ca structură
  Given un DispatchPlan complet (PLAN 8.9)
  Then trece Bean Validation
```

```
Scenario: răspuns invalid de la model
  Given JSON incomplet
  Then planul e respins controlat (nu 500)
```

Tasks: - [] `agent/dtos/DispatchPlan` + `BatteryCommand` (Bean Validation) - [] respingere controlată pe răspuns invalid - [] unit

DoD: invalid → respins, nu excepție.

[F11-S2] Simulate & validate (dry-run)

Prioritate: High · **Points:** 5 · **Dep:** F11-S1, F05-S3, F08-S2

Ca sistem, vreau să simulez și validez planul fără efecte, **ca să** prind planuri periculoase înainte de execuție.

Acceptance (Gherkin):

Scenario: dry-run fără persistare

When simulateDispatchPlan(plan)

Then rulează prin EnergyEngine + GridProtection fără a scrie în DB

Scenario: plan malițios

Given comandă peste limite

When validateDispatchPlan(plan)

Then respins + auditat

Tasks: - [] agent/service/queryService/simulateDispatchPlan - [] validateDispatchPlan (prin BatteryCommandValidator + transformator) - [] unit plan valid / malițios

DoD: scenariile PLAN 15.4 verzi.

[F11-S3] Endpoints planuri (aprobare/execuție)

Prioritate: High · **Points:** 3 · **Dep:** F11-S2

Ca OPERATOR, **vreau** să aprob/resping planuri; în ADVISORY execuția e blocată, **ca să** păstrez controlul uman.

Acceptance (Gherkin):

Scenario: execute în ADVISORY

Given mode=ADVISORY

When POST /agent/plans/{id}/execute

Then 409 AGENT_MODE_ADVISORY (nu execută)

Scenario: provider indisponibil

Given LLM down

Then simularea continuă, niciun plan nou (degradare sigură)

Tasks: - [] GET /agent/plans , /{id} , /{id}/tool-calls @PreAuthorize('AGENT_VIEW') - [] POST /plans/{id}/approve , /reject , /execute , /agent/emergency-stop @PreAuthorize('AGENT_OPERATE') - [] 409 în ADVISORY; SUPERVISED → același validator + CommandExecution - [] test scenariu PLAN 15.4

DoD: ADVISORY nu execută · degradare sigură.

EPIC 12 — Audit & rapoarte

[F12-S1] Evenimente & rapoarte agregate

Prioritate: Med · **Points:** 3 · **Dep:** toate

Ca ADMIN/OPERATOR, **vreau** audit și rapoarte pe intervale, **ca să** evaluez sistemul și agentul.

Acceptance (Gherkin):

```
Scenario: raport energie
  When GET /reports/energy cu AUDIT_VIEW
  Then 200 cu agregări pe interval (import/export, profit, planuri acc/resp, token usage)
```

Tasks: - [] GET /audit/events , /reports/energy|financial|agent-performance
@PreAuthorize('AUDIT_VIEW') - [] agregări pe intervale

DoD: DoD global.

EPIC 13 — Dashboard live (WebSocket)

[F13-S1] Evenimente live STOMP

Prioritate: Med · **Points:** 3 · **Dep:** F04-S3, F08-S2, F10-S3

Ca frontend, **vreau** evenimente la fiecare tick, **ca să** afișez dashboard-ul live — **fără** comenzi critice pe WebSocket.

Acceptance (Gherkin):

```
Scenario: stare live
  Given client STOMP abonat la /topic/community/state
  When trece un tick
  Then primește CommunityStateUpdated

Scenario: WebSocket cade
  Given WebSocket oprit
  Then protecția și comenzile rulează în continuare
```

Tasks: - [] `system/config/WebSocketConfig` (STOMP, `/ws`, broker `/topic`) - [] publicare din tick/servicii (`SimpMessagingTemplate`) — **DTO-uri, nu entități** - [] test client STOMP

DoD: doar evenimente, nu comenzi critice.

EPIC 14 — Rulare completă în Docker Compose

 **Înveți aici — Docker.** Dockerfile multi-stage + compose full-stack. `depends_on` + `healthcheck`, env din `.env` (gitignored), `host` = numele serviciului (`jdbc:mysql://mysql:3306/solarsync`).

[F14-S1] Dockerfile multi-stage

Prioritate: Med · **Points:** 2 · **Dep:** F01-S2

Ca dezvoltator, **vreau** o imagine a backend-ului, **ca să** rulez aplicația în container.

Acceptance (Gherkin):

```
Scenario: build image
  When docker build .
  Then rezultă o imagine cu JRE slim care pornește jar-ul pe 8080
```

Tasks: - [] `Dockerfile` (build Maven → runtime `eclipse-temurin:21-jre`, `EXPOSE 8080`) - [] `.dockerignore`

DoD: imaginea pornește local.

[F14-S2] Compose full-stack

Prioritate: High · **Points:** 3 · **Dep:** F14-S1

Ca comisie, **vreau** `docker compose up` să ridice tot, **ca să** demonstrez cu o singură comandă (criteriul #1 MVP).

Acceptance (Gherkin):

Scenario: pornire completă

Given o mașină curată cu .env completat

When docker compose up --build

Then MySQL healthy, backend pornit după el, Flyway aplică V1..V9, swagger accesibil

Scenario: reset

When docker compose down -v

Then volumul DB e șters

Tasks: - [] serviciu `backend` (`build: .` , `depends_on: mysql service_healthy` , `env_file: .env`) - [] `.env.example` (committed) + `.env` în `.gitignore` - [] verify pe mașină curată

DoD: o singură comandă ridică stack-ul.

Definition of Done (global, pe FIECARE story)

- ☐ `./mvnw clean compile` → BUILD SUCCESS, fără warning-uri noi
- ☐ schema schimbată DOAR prin migrație Flyway nouă (`ddl-auto: validate` trece)
- ☐ DTO request/response (nu entitate expusă), `@Valid` pe input
- ☐ serviciu `command` / `query` (interfață + `Impl`), mapper static, excepții cu mesaj din `ErrorConstants`
- ☐ `@PreAuthorize("hasAuthority('...')")` pe endpoint sensibil; owner check în service unde e `USER`
- ☐ erori prin `GlobalExceptionHandler` (`ApiErrorResponse`), texte în engleză
- ☐ teste: unit pentru logica deterministă, integrare pentru endpoint-uri securizate
- ☐ commit: `git commit -m "[F<epic>-S<nr>] <descriere>"`

Ordine recomandată

- EPIC 00–03** — fundație + auth + case/baterii. Teren cunoscut.
- EPIC 04** — simularea (tick reproductibil + telemetrie).
- EPIC 05–09** — inima deterministă. **Nu trece mai departe fără testele verzi.**
- EPIC 10–11** —  agentul în ADVISORY (mock întâi, Spring AI real la Modul 8). Contribuția nouă.
- EPIC 12–13** — audit/rapoarte + WebSocket.
- EPIC 14** —  Docker: pornire cu o singură comandă pentru demo.



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro